THE MYSTERIOUS CIPHER

Course: System Skills

Assignment: 3

Author: Srisupha Chawla

Student ID: 6680520

1. Executive Summary

This report presents the reverse-engineering analysis of a compiled x86-64 assembly program implementing a decryption routine. Through static analysis of the assembly's data and text sections, the secret mathematical key was identified as **-3** (represented in 8-bit two's complement as 0xFD or 253).

By analyzing the decryption loop mechanics, the cipher was determined to be a Caesar-style shift cipher where each ciphertext character is shifted forward by 3 positions ($C - (-3) = C + 3$). Applying this transformation to the hardcoded ciphertext **"ddbw"** successfully recovered the original plaintext: **"ggez"**.

2. Architectural Static Analysis

2.1 The Data Section

The program initializes three variables in the .data memory segment:

- ciphertext: A null-terminated ASCII string (.asciz) containing the encrypted bytes "ddbw".
- num1: A 32-bit signed long integer (.long) with a value of 2.
- num2: A 32-bit signed long integer (.long) with a value of -5.

2.2 Register Setup & Key Computation

Before entering the loop, the program computes the decryption key using the following instructions:

```
None
lea ciphertext(%rip), %rdi  # Load address of ciphertext into %rdi
movl num1(%rip), %eax       # %eax = num1 (2)
movl num2(%rip), %ecx       # %ecx = num2 (-5)
addl %ecx, %eax             # %eax = %eax + %ecx -> 2 + (-5) = -3
movb %al, %bl               # %bl = lowest byte of %eax (-3 / 0xFD)
```

1. **Addition:** The system adds num1 (2) and num2 (-5) using addl, storing the sum -3 in register %eax.
2. **Truncation:** The movb %al, %bl instruction extracts the lowest 8 bits (one byte) of register %eax and saves it into %bl.
3. **Two's Complement Representation:** In standard 8-bit two's complement representation, the signed integer -3 is represented as:
   $$256 - 3 = 253 = \text{0xFD} = 11111101_2$$
   Thus, the decryption key stored in %bl is **-3** (or 0xFD).

3. Decryption Loop Mechanics

Once the key is calculated, the program enters the decrypt_loop label to decode the string in-place:

Code snippet

```
None
decrypt_loop:
    movzbq (%rdi), %rax     # Load the current character into %rax, zero-extended
    testb %al, %al          # Compare character byte to 0 (null terminator '\0')
    je decrypt_done         # If 0, exit loop
    subb %bl, %al           # %al = %al - %bl (Subtract the key)
    movb %al, (%rdi)        # Overwrite character in memory with decrypted value
    inc %rdi                # Increment pointer to the next character
    jmp decrypt_loop        # Repeat loop
```

3.2 The Mathematical Transformation

The core decryption operation is subb %bl, %al. This subtracts the key stored in %bl from the ciphertext character in %al:

$$\text{Plaintext Byte} = \text{Ciphertext Byte} - \text{Key}$$
Substituting our computed key of -3 into the equation yields:

$$\text{Plaintext Byte} = \text{Ciphertext Byte} - (-3)$$
$$\text{Plaintext Byte} = \text{Ciphertext Byte} + 3$$
Because subtracting -3 is mathematically identical to adding 3, the program decrypts the string by shifting the ASCII value of each character **forward by 3 positions**.

4. Plaintext Recovery and ASCII Mapping

Applying the $C + 3$ transformation to each byte of the ciphertext "ddbw" resolves the original string:

Ciphertext Char	Hex Value	Decimal ASCII	Decryption Operation	Decrypted ASCII	Decrypted Char
d	0x64	100	$100 + 3$	103	**g**
d	0x64	100	$100 + 3$	103	**g**
b	0x62	98	$98 + 3$	101	**e**
w	0x77	119	$119 + 3$	122	**z**
\0	0x00	0	*Loop Terminates*	0	**\0**

The program then executes call puts using the reloaded address in %rdi to print the final, decrypted plaintext to stdout: **ggez**.

5. Conclusion

The "Mysterious Cipher" is an elegant implementation of a Caesar shift cipher optimized for minimal assembly-level execution. By manipulating signed integers (num1 and num2) and utilizing a subtraction instruction (subb) against a negative key, the developer cleverly obfuscated what is functionally a simple

addition cipher. This exercise demonstrates how compiler representations of signed integers in two's complement form can be leveraged to streamline arithmetic operations on modern processors.